convergence guarantees has been recently proposed and implemented in a easy-to-use software tool Carriere et al. (2020). With a different perspective, another theoretical framework to study the differentiable structure of functions of persistence diagrams has been proposed in Leygonie et al. (2019).

7 TDA for data sciences with the GUDHI library

In this section we illustrate TDA methods with the Python library Gudhi[11] (Maria et al., 2014) together with popular libraries as numpy (Walt et al., 2011), scikit-learn (Pedregosa et al., 2011), pandas (McKinney et al., 2010). More illustrations with python notebooks can be found in the Tutorial Github page[12] of Gudhi.

7.1 Bootstrap and comparison of protein binding configurations

This example is borrowed from Kovacev-Nikolic et al. (2016). In this paper, persistent homology is used to analyze protein binding and more precisely it compares closed and open forms of the maltose-binding protein (MBP), a large biomolecule consisting of 370 amino acid residues. The analysis is not based on geometric distances in $\mathbb{R}^3$ but on a metric of *dynamical distances* defined by

$$D_{ij} = 1 - |C_{ij}|,$$

where C is the correlation matrices between residues. The data can be download at this link[13].

```
import numpy as np
import gudhi as gd
import pandas as pd
import seaborn as sns

corr_protein = pd.read_csv("mypath/1anf.corr_1.txt",
                           header=None,
                           delim_whitespace=True)
dist_protein_1 = 1- np.abs(corr_protein_1.values)
rips_complex_1= gd.RipsComplex(distance_matrix=dist_protein_1,
                               max_edge_length=1.1)
simplex_tree_1 = rips_complex_1.create_simplex_tree(max_dimension=2)
diag_1 = simplex_tree_1.persistence()
gd.plot_persistence_diagram(diag_1)
```

For comparing persistence diagrams, we use the bottleneck distance. The block of statements given below computes persistence intervals and computes the bottleneck distance for 0-homology and 1-homology:

```
interv0_1 = simplex_tree_1.persistence_intervals_in_dimension(0)
interv0_2 = simplex_tree_2.persistence_intervals_in_dimension(0)
bot0  = gd.bottleneck_distance(interv0_1,interv0_2)

interv1_1 = simplex_tree_1.persistence_intervals_in_dimension(1)
interv1_2 = simplex_tree_2.persistence_intervals_in_dimension(1)
bot1  = gd.bottleneck_distance(interv1_1,interv1_2)
```

In this way, we can compute the matrix of bottleneck distances between the fourteen MPB. Finally, we apply a multidimensional scaling method to find a configuration in $\mathbb{R}^2$ which almost match with the bottleneck distances, see Figure 13(c). We use the scikit-learn library for the MDS:

```
import matplotlib.pyplot as plt
from sklearn import manifold
```

[11] http://gudhi.gforge.inria.fr/python/latest/
[12] https://github.com/GUDHI/TDA-tutorial
[13] https://www.researchgate.net/publication/301543862_corr

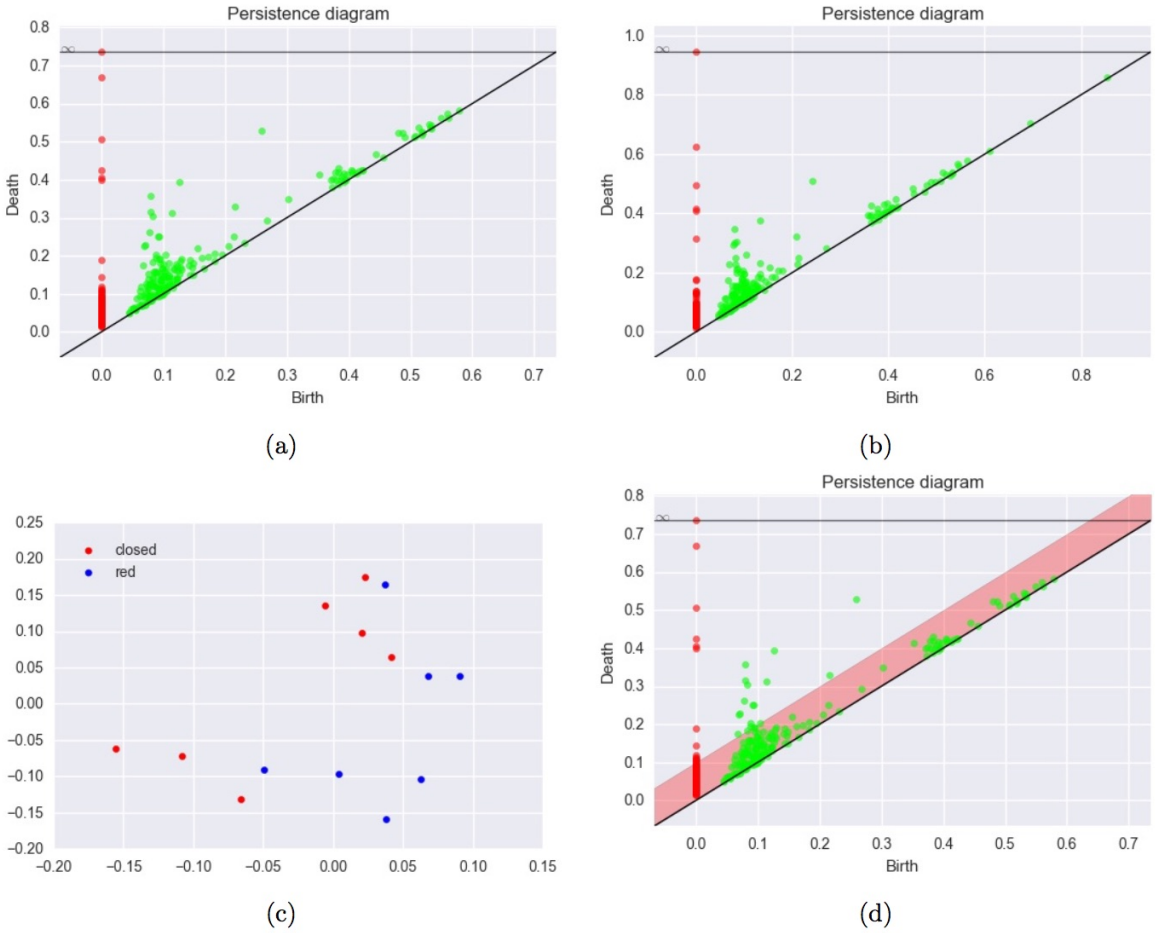


Figure 13: (a) and (b) two persistence diagrams for two configurations of MBP. (c) MDS configuration for the matrix of bottleneck distances. (d) Persistence diagram and confidence region for the persistence diagram of a MBP.

```

mds = manifold.MDS(n_components=2, dissimilarity="precomputed")
config = mds.fit(M).embedding_

plt.scatter(config [0:7,0], config [0:7, 1], color='red', label="closed")
plt.scatter(config [7:l,0], config [7:l, 1], color='blue', label="red")
plt.legend(loc=1)
```

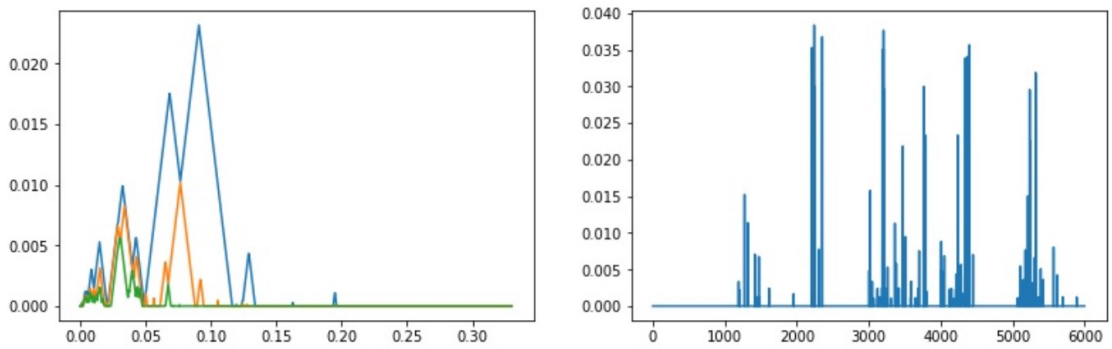
We now define a confidence band for a diagram using the bottleneck bootstrap approach. We resample over the lines (and columns) of the matrix of distances and we compute the bottleneck distance between the original persistence diagram and the bootstrapped persistence diagram. We repeat the procedure many times and finally we estimate the quantile 95% of this collection of bottleneck distances. We take the value of the quantile to define a confidence band on the original diagram (see Figure 13(d)). However, such a procedure should be considered with caution because as far as we know the validity of the bottleneck bootstrap has not been proved in this framework.

7.2 Classification for sensor data

In this experiment, the 3d acceleration of 3 walkers (A, B and C) have been recorded from the sensor of a smart phone[14]. Persistence homology is not sensitive to the choice of axes and so no preprocessing is necessary to align the 3 times series according to the same axis. From these three times series, we have picked at random sequences of 8 seconds in the complete time series, that is 200 consecutive points of acceleration in $\mathbb{R}^3$. For each walker, we extract 100 time series in this way. The next block of statements computes the persistence for the alpha complex filtration for `data_A_sample`, one of the 100 times series of acceleration of Walker A.

```
alpha_complex_sample = gd.AlphaComplex(points = data_A_sample)
simplex_tree_sample = alpha_complex_sample.create_simplex_tree(max_alpha_square
    =0.3)
diag_Alpha = simplex_tree_sample.persistence()
```

From `diag_Alpha` we can then easily compute and plot the persistence landscapes, see Figure 14(a). For all the 300 times series, we compute the persistence landscapes for dimension 0 and 1 and we compute the three first landscapes for the 2 dimensions. Moreover, each persistence landscape is discretized on 1000 points. Each time series is thus described by 6000 topological variables. To predict the walker from these features, we use a random forest (Breiman, 2001), which is known to be an efficient in such an high dimensional setting. We split the data into train and test samples at random several times. We finally obtain an averaged classification error around 0.95. We can also visualize the most important variables in the Random Forest, see Figure 14(b).



(a) (b)

Figure 14: (a)The three first landscapes for 0-homology of the alpha shape filtration defined for a time series of acceleration of Walker A. (b) Variable importances of the landscapes coefficients for the classification of Walkers. The 3 000 first coefficients correspond to the three landscapes of dimension 0 and the 3 000 last coefficients to the three landscapes of dimension 1. There are 1000 coefficients per landscape. Note that the first landscape of dimension 0 is always the same using the Rips complex (a trivial landscape) and consequently the corresponding coefficients have a zero importance value.

Acknowledgements This work was partly supported by the French ANR chair in Artificial Intelligence TopAI. We thank the authors of Kovacev-Nikolic et al. (2016) for making their data available.

[14]The dataset can be download at this link `http://bertrand.michel.perso.math.cnrs.fr/Enseignements/TDA/data_acc`